1 Paper review

Sensor failures and transmission losses leave gaps in multivariate time series. Interpolation, neighbouring-sensor averages and matrix factorisation fill these gaps using patterns over time or similarities between channels. Recurrent methods such as BRITS also use relationships between channels, but do not explicitly model the sensor network. GRIN represents this network as a graph, allowing it to learn nonlinear relationships between connected sensors as well as patterns over time *[1, Sections 1–2].*

Preliminaries and the GRIN idea

At time t, each of N identified sensors has d measurements, stored in $X_t \in \mathbb{R}^{Nxd}$. The fixed weighted adjacency matrix W describes the connections between sensors. The binary mask M_t has the same shape as X_t: 1 means observed and 0 means missing. GRIN first predicts missing readings from previous memory, then uses neighbouring sensors to refine them and update that memory. A second pass runs backwards through the sequence. Combining the two passes uses information from both sides of a gap (see Figure 1) *[1, Sections 3–4].*

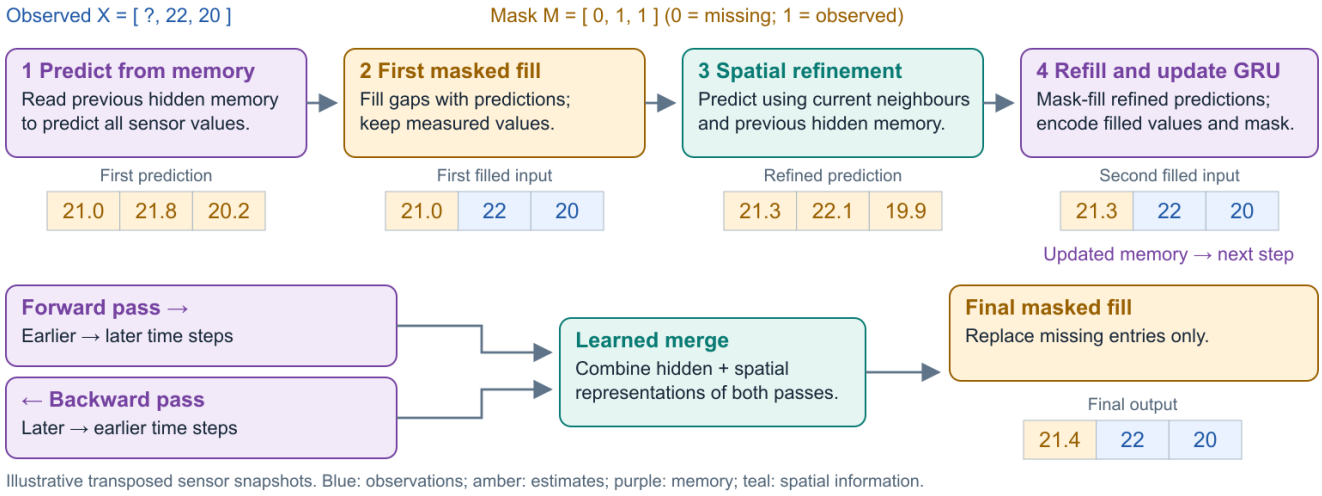


Figure 1 GRIN processing steps with example values. Each row lists sensors A–C horizontally; for d = 1, this is a transposed X_t snapshot. The mask stays the same and observed readings are kept.

Temporal prediction and selective filling

The first readout uses previous memory H_{t-1} to predict the current measurements. This memory contains information from earlier readings and their spatial relationships. The filler Φ keeps readings where the mask is 1 and inserts predictions where it is 0. The mask is passed on with the filled data, so later steps can still distinguish observations from estimates *[1, Eqs. 7–8].*

$$\hat{Y}_t^{(1)} = H_{t-1}V_h + b_h; \quad \Phi(Y_t) = \underbrace{M_t \odot X_t}_{\text{keep observed}} + \underbrace{(1 - M_t) \odot Y_t}_{\text{fill missing}}$$

Spatial refinement through message passing

Each neighbour contributes a learned message built from its filled readings, mask and previous memory. The messages are summed and combined with the target sensor's previous memory to form S_t. The target's current reading is excluded, so the model cannot simply copy it. A second readout uses S_t and previous memory to produce a refined prediction. Applying Φ again replaces only missing readings *[1, Eqs. 9–10].*

$$\hat{Y}_t^{(2)} = \underbrace{[S_t \| H_{t-1}]}_{\text{neighbours and prior memory}} V_s + b_s; \quad \hat{X}_t^{(2)} = \underbrace{\Phi(\hat{Y}_t^{(2)})}_{\text{fill only missing entries}}$$

The readout turns the information from neighbouring sensors and previous memory into a new estimate. The mask then determines which readings are replaced.

Memory update

A message-passing GRU updates memory using the refined, filled readings, their masks and previous memory. The reset gate controls how much past memory enters the candidate c_t. The update gate u_t determines how much of this candidate is combined with retained memory. Both gates use graph information, so neighbouring sensors can influence the update. The resulting H_t is used at the next time step *[1, Eqs. 3–6]*.

$$h_t^i = \underbrace{u_t^i \odot h_{t-1}^i}_{\text{retained memory}} + \underbrace{(1 - u_t^i) \odot c_t^i}_{\text{candidate contribution}}$$

Bidirectional reconstruction

The backward module runs the same steps from the end of the sequence, so its memory contains information from later times. An MLP combines the spatial representations and memories from both passes to predict the missing readings. The final application of Φ keeps all observed values *unchanged [1, Eq. 11].*

$$\hat{y}_t^i = MLP([\ \underbrace{s_t^{i,fwd} \| h_{t-1}^{i,fwd}}_{\text{forward information}} \ \| \ \underbrace{s_t^{i,bwd} \| h_{t+1}^{i,bwd}}_{\text{backward information}} \ ])$$

In short: GRIN first estimates missing readings from memory, improves them using neighbouring sensors and updates its memory for the next step. Combining forward and backward passes uses information on both sides of a gap, while the mask keeps observed measurements unchanged.

Evaluation and reported performance

The paper compares GRIN with statistical and neural methods using MAE, MSE and mean relative error, averaged over five runs. Evaluation readings are hidden from the model. Air-quality experiments cover in-sample and out-of-sample reconstruction. Traffic and energy experiments use 25% random point losses or a combination of random losses and longer outages. Training uses the final and intermediate predictions from both directions *[1, Section 5; Appendix A].*

Table 1 Out-of-sample MAE for GRIN and the best competing method in each setting; lower is better. Reduction = 100 × (baseline − GRIN) / baseline. Values are means from [1, Tables 1–2]; standard deviations are omitted. Units differ between datasets, so values should be compared within each row.

Dataset and setting	Best other model	Other MAE	GRIN MAE	Reduction
AQI-36 · air quality	BRITS	14.50	12.08	16.7%
AQI · air quality	MPGRU	18.76	14.73	21.5%
PEMS-BAY · blocks	MPGRU	1.59	1.14	28.3%
PEMS-BAY · points	MPGRU	1.11	0.67	39.6%
METR-LA · blocks	BRITS	2.34	2.03	13.2%
METR-LA · points	BRITS	2.34	1.91	18.4%
CER-E · blocks	MPGRU	0.53	0.42	20.8%
CER-E · points	MPGRU	0.41	0.29	29.3%

GRIN has the lowest reported MAE in these settings. Ablation experiments show that spatial decoding and bidirectional processing are particularly useful for blocks of missing data. Diffusion convolution restricts message passing to local neighbourhoods. The virtual-sensing experiment also shows that GRIN can estimate readings for an entirely unobserved sensor when its neighbours provide enough information *[1, Sections 5.1–5.2; Appendix A].*

Conclusion

GRIN improves imputation across the tested datasets by using relationships between sensors. Its performance still depends on useful connections and representative training data. The bidirectional model needs future observations, which limits immediate online use. Accuracy guarantees and adaptation to non-stationary data require further work *[1, Section 6].*

2 Reproducing the results

2.1 The original code

The AQI-36 out-of-sample experiment was chosen because it is one of the paper's main benchmarks and includes the authors' evaluation mask. The original GRIN model was downloaded from commit 4a28afbb092600b6e6abeeabaaf67e87dbd1ed6e. Its files were checked using SHA-256. Training was run in PyTorch instead of the original Lightning setup to support Python 3.12. The model code was left unchanged.

2.2 Code implementation

The Jupyter notebook downloads the data, builds the graph, trains GRIN and evaluates the best validation checkpoint. AQI-36 contains 8,759 hourly observations at 36 stations. March, June, September and December are used for testing. Validation windows are placed before these months, and overlapping training windows are removed. This gives 4,948 training, 564 validation and 2,788 test windows with no shared observations. The mean and standard deviation are calculated from visible training data only. Withheld validation and test readings remain hidden from the model.

Execution parameters

Parameter group	Values used
Data and windows	AQI-36; 36 hours; stride 1; validation fraction 0.10; stored evaluation mask
Graph	Geographic Gaussian kernel; threshold 0.10; no self-loops; 654 directed entries
Architecture	Hidden 64; feed-forward 64; node embedding 8; one recurrent layer; MLP merge
Graph operations	Encoder diffusion order 2; decoder order 1; global attention off; no exogenous inputs
Regularisation	Dropout 0; feed-forward dropout 0; layer normalisation off; weight decay 0
Training budget	30 epochs; 512 windows per epoch sampled with replacement; batch size 32
Optimisation	Adam; learning rate 0.001; cosine decay to 0.0001 over 30 epochs; gradient norm clip 5
Loss and masking	Sum of five scaled masked MAEs; intermediate loss weight 1; additional masking 0.05
Selection and randomness	Minimum validation window MAE; patience 40; seed 42; deterministic PyTorch operations
Runtime	CPU; one PyTorch thread; Python 3.12.14; PyTorch 2.14.1+cpu; exact dependencies in repository
Final scoring	Average overlapping test predictions by timestamp, then score the fixed held-out targets

The paper uses up to 300 epochs and 5,120 windows per epoch, repeated over five runs. The CPU run uses fewer training steps and a shorter cosine learning-rate schedule. The notebook also includes the paper's training settings.

2.3 Comparison to the paper's result

Training took 13.3 minutes for 30 epochs and 480 optimiser updates. Epoch 18 had the lowest validation error, so its checkpoint was used to predict the 20,434 withheld test readings. The table compares this run with the paper and two local baselines. Errors use the original PM2.5 scale; lower is better. The paper reports averages over five runs, while the local results use one seed.

Method	Result source	MAE	MSE
GRIN	Local reduced-budget run	12.789	501.39
GRIN	Paper, out-of-sample	12.08 ± 0.47	523.14 ± 57.17
BRITS	Paper, out-of-sample	14.50 ± 0.35	662.36 ± 65.16
Station mean	Local training-only control	55.184	4721.55
Linear interpolation	Local bidirectional control	14.598	691.16

The local GRIN MAE is 0.709 $\mu g/m^3$ (5.9%) higher than the published mean and 12.4% lower than the local interpolation baseline. Its mean relative error is 18.36%, compared with 17.00 ± 0.67% in the paper. The local baselines use the same test readings as GRIN, but were implemented separately from the paper's baseline code.

The local MSE is lower than the published GRIN mean, even though its MAE is higher. The results therefore depend on the error metric. One run with less training is not enough to conclude that the model performs better overall.

The main difference is the amount of training: 480 updates here, compared with up to 48,000 per run in the paper. Only one seed was tested, so variation between runs could not be measured. The original seeds were not published. Different software versions and the shorter learning-rate schedule may also affect the results. Further runs would be needed to separate these effects.

The notebook checks that the data splits share no timestamps, withheld validation and test readings stay out of the inputs, and observed values remain unchanged after imputation. The tests also cover the five-part loss, output shapes and averaging of overlapping predictions. SHA-256 hashes are stored for the data, model files and saved results.

Reproducing the experiment

The notebook, code, settings and saved model are available in the public repository. It also contains the training history and predictions used for this comparison. The README gives the installation and training commands. The data and original GRIN files are downloaded automatically and checked against their recorded hashes.

https://github.com/Achiee7/DW-Assignement-2--Implementing-GRIN-for-imputation

Notebook: notebooks/grin_aqi36_reproduction.ipynb. From the repository root, run python scripts/run_notebook.py using Python 3.12. Add --retrain to train again, or --profile paper --seed 42 to use the paper's training budget for one seed. Repeat with five seeds for a five-run comparison.

[1] Cini, A., Marisca, I. and Alippi, C. (2022). Filling the G_ap_s: Multivariate Time Series Imputation by Graph Neural Networks. ICLR. https://arxiv.org/abs/2108.00298v3.